

4-Hour Production-Level Assignment

Node.js + MongoDB Aggregation Pipeline + JWT

Project: E-Commerce Analytics & Order Management API | Level: Intermediate → Production-Oriented | Duration: 4 Hours

Technology Stack: Node.js, Express.js, MongoDB, Mongoose, JWT, bcrypt, Joi, Helmet, CORS, express-rate-limit

Assessment focus: secure REST APIs, JWT authentication/RBAC, realistic order modeling, and advanced MongoDB aggregation pipelines.

1. Project Scenario

Build an E-Commerce Analytics & Order Management API. Authenticated users can register, log in, browse products, create orders, and view their orders. Admin users can create products and access business analytics generated primarily through MongoDB aggregation pipelines.

2. Four-Hour Execution Plan

Time	Task	Expected Output
0:00-0:30	Project setup + MongoDB	Express app, database connection, environment configuration
0:30-1:15	Authentication + JWT	Register, login, JWT middleware, RBAC
1:15-1:45	Product + Order APIs	Core CRUD/read and order creation
1:45-3:15	Aggregation pipelines	Dashboard, category, products, customers, monthly/date analytics
3:15-3:45	Security + validation	Joi, Helmet, CORS, rate limiting, error handling
3:45-4:00	Testing + documentation	Postman collection, README, evidence

3. Required Project Structure

```
ecommerce-analytics-api/  
  ■■■ src/config/db.js  
  ■■■ src/controllers/{auth,product,order,analytics}.controller.js  
  ■■■ src/models/{User,Product,Order}.js  
  ■■■ src/routes/{auth,product,order,analytics}.routes.js  
  ■■■ src/middleware/{auth,error}.middleware.js  
  ■■■ src/services/analytics.service.js  
  ■■■ src/utils/jwt.js  
  ■■■ src/app.js  
  ■■■ server.js  
  ■■■ .env  
  ■■■ .env.example  
  ■■■ README.md
```

4. Database Models

User

```
{ name, email, password, role: 'customer'|'admin', isActive, createdAt }
```

Requirements: unique email, bcrypt password hashing, and never return password in API responses.

Product

```
{ name, category, price, stock, isActive, createdAt }
```

Order

```
{ user: ObjectId, items: [{ product: ObjectId, quantity: Number, price: Number }], totalAmount: Number,  
status: 'pending'|'confirmed'|'shipped'|'delivered'|'cancelled', createdAt }
```

5. Authentication & Authorization

Register: POST /api/auth/register

```
{ "name": "Rahul", "email": "rahul@gmail.com", "password": "Password@123" }
```

Implement email validation, duplicate-email checking, bcrypt hashing, proper status codes, and centralized error handling.

Login: POST /api/auth/login

```
{ "email": "rahul@gmail.com", "password": "Password@123" }
```

Return a JWT access token. JWT payload should contain userId and role. Protected requests use Authorization: Bearer <token>.

JWT middleware must: extract Bearer token, verify it, attach user information to req.user, and reject invalid/expired tokens.

RBAC: customers can place orders; admins can create products and access analytics.

6. Product APIs

- POST /api/products — Admin only. Create a product with name, category, price, and stock.
- GET /api/products — Support pagination, search, category filter, minimum/maximum price, and sorting.
- Example: GET
/api/products?category=Laptop&minPrice;=500000&maxPrice;=1500000&page;=1&limit;=10

7. Order APIs

POST /api/orders — Authenticated users only.

```
{ "items": [{ "product": "PRODUCT_ID", "quantity": 2 }, { "product": "PRODUCT_ID_2", "quantity": 1 }] }
```

Server-side requirements: validate products, check stock, fetch current product prices, calculate totalAmount on the server, and never trust client-supplied price or totalAmount.

GET /api/orders — Return the authenticated user's orders.

8. MongoDB Aggregation — Main Assessment

The aggregation section is the primary evaluation area. Students must be able to explain every pipeline stage they use.

API	Required Result	Required Concepts
/api/analytics/dashboard	Total orders, revenue, average order value, cancelled, complete, group, sum, \$avg, \$cond	\$match, \$group, \$sum, \$avg, \$cond
/api/analytics/category-sales	Orders, quantity and revenue by product category	\$unwind, \$lookup, \$group, \$project, \$sort
/api/analytics/top-products	Top 5 products by revenue and quantity sold	\$unwind, \$lookup, \$group, \$sort, \$limit
/api/analytics/customers	Customer name/email, total orders and total spending	\$group, \$lookup, \$unwind, \$project, \$sort
/api/analytics/monthly-revenue	Revenue grouped by year and month	\$group, \$year, \$month, \$sum, \$sort
/api/analytics/sales?from=&to=	Orders, revenue, average order value, total items for date range	\$match, \$group, date filtering
/api/analytics/order-status	Order count and revenue by status	\$group, \$sum, \$sort

9. Aggregation Pipeline Expectations

Category sales flow: Orders → \$unwind items → \$lookup products → \$unwind product → \$group by category → \$project → \$sort.

Customer report flow: Orders → \$group by user → \$lookup users → \$unwind → \$project → \$sort.

Top products: unwind order items, join product data, group by product, calculate quantity/revenue, sort descending, limit to 5.

10. Production Security & Quality Requirements

- Use environment variables: PORT, MONGO_URI, JWT_SECRET, JWT_EXPIRES_IN.
- Provide .env.example; never commit secrets.
- Use Joi for request validation.
- Use Helmet and CORS.
- Rate-limit register/login endpoints.
- Centralize error handling.
- Handle invalid ObjectId, duplicate keys, validation errors, JWT errors, authorization errors, and unexpected server errors.
- Use authentication and authorization separately.

11. Indexing & Aggregation Optimization

Students must identify appropriate indexes and explain why each one exists.

```
UserSchema.index({ email: 1 });
ProductSchema.index({ category: 1, price: 1 });
OrderSchema.index({ user: 1, createdAt: -1 });
OrderSchema.index({ status: 1, createdAt: -1 });
```

Run `explain('executionStats')` and document `executionTimeMillis`, `totalDocsExamined`, and `totalKeysExamined` before/after indexing where practical.

12. Sample Data Requirement

Minimum test dataset: 10 users, 20 products, 50 orders, and 100+ order items. Include multiple categories, customers, prices, quantities, statuses, and order dates so that aggregation results are meaningful.

13. Postman Collection

Create a Postman collection containing:

- 01 Register
- 02 Login
- 03 Create Product
- 04 Get Products
- 05 Create Order
- 06 Get Orders
- 07 Dashboard
- 08 Category Sales
- 09 Top Products
- 10 Customer Analytics
- 11 Monthly Revenue
- 12 Date Sales
- 13 Order Status

14. Bonus Challenges

- Add pagination to analytics endpoints.
- Support dynamic sorting such as revenue or quantity.
- Add reusable date filters to dashboard analytics.
- Use `$facet` to return summary, topProducts, categorySales, and orderStatus from a single aggregation request.

15. Evaluation — 100 Marks

Area	Marks
Project structure	10

Area	Marks
JWT authentication	15
Authorization / RBAC	10
Product & Order APIs	10
MongoDB aggregation pipelines	25
Validation & error handling	10
Security	5
Indexing & optimization	5
Postman testing	5
Code quality & documentation	5
TOTAL	100

16. Final Deliverables

- GitHub repository with complete source code.
- .env.example file.
- README.md with setup, API list, and architecture explanation.
- Postman collection.
- MongoDB seed/sample data.
- Aggregation pipeline explanations.
- API testing screenshots or evidence.
- Index/explain performance report.

17. Viva / Interview Questions

1. Why should aggregation happen in MongoDB instead of Node.js?
2. What is the difference between MongoDB \$lookup and a SQL JOIN?
3. Why is \$unwind required for order items?
4. Why should \$match generally be placed as early as possible?
5. What is the purpose of \$facet?
6. How does an index improve aggregation performance?
7. What happens when a JWT expires?
8. What is the difference between authentication and authorization?
9. Why should the order price be stored instead of always reading the current product price?
10. Why should the client not send totalAmount?
11. How would you implement refresh-token rotation and reuse detection in a production system?

Instructor Note: The assessment is intentionally designed so that students must combine API engineering, security, data modeling, and MongoDB aggregation rather

than building only basic CRUD.